FPGA Tactical Controller for StarCraft II Micro Scenarios

Target Tier: Tier A (100%) **Course:** EECS 3216 (Fall 2026) **Topic:** Datapath / Control / VGA

Group Members: Marlin Lachance (213885025), Vaughn Chan (219846914), Hafsa Manazir (219780428), Vladislav Prasko (220741781)

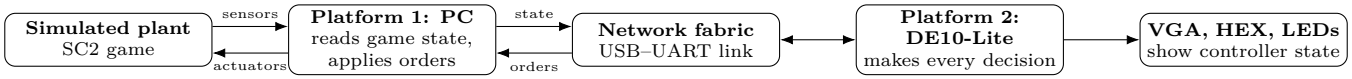
[built] working on the board; shown to the TA during scoping to justify this proposal. [proposed] new work for the final demonstration.

1. Overview and Difficulty Justification

The FPGA commands Marines against waves of Roaches in StarCraft II (SC2); the PC only relays game state and orders.

- [built] On DeepMind's *DefeatRoaches* mini-game [1], the FPGA chooses which Roach each Marine attacks, and VGA draws the units and target lines.
- [proposed] In our own variant, the Marines start as two separate squads. Using Lanchester's square law, the FPGA decides when a squad is too weak to fight alone, regroups it with the other squad, and reengages together.

Tier A: a custom UART link, pipelined targeting datapath, control FSMs and VGA display run on the board at once.



The structure follows Lee and Seshia's cyber-physical system, Fig. 1.1 [2], with the SC2 game in place of a physical plant.

2. Target Hardware and Peripherals (DE10-Lite Only)

- **Onboard:** MAX 10 FPGA with 50 MHz clock; GPIO header (UART); VGA port; SW0 (targeting policy); SW4:1 (inspect a Marine); KEY0 (reset); HEX5:0; LEDR9:0.
- **External:** 3.3 V USB-to-UART adapter; VGA monitor; PC running SC2 with our Python bridge.

3. Combinational and Sequential Logic Architecture

- **Combinational:**
  - [built] Squared Marine–Roach distances, attack-range checks, target-ranking comparators, packet checksum.
  - [proposed] Squad strengths $P = kN^2$ (square lookup table and multiplier) and strength comparators.
- **Sequential:**
  - [built] UART FSMs; snapshot registers; ranking pipeline (one Marine–Roach pair per clock); VGA sync counters.
  - [proposed] One squad FSM per squad (ATTACK, REGROUP, HOLD).

4. Proposed Extension: Lanchester Regroup-and-Reengage

When every unit can fire, Lanchester's square law gives a force strength $P = kN^2$ [4], where N is the number of living units and k is a fixed value per unit type. Two squads of 5 Marines fighting apart total $2 \cdot 5^2k = 50k$; together they have $10^2k = 100k$. The controller uses this to avoid being defeated one squad at a time.

- **ATTACK:** the squad fights while its own strength is at least the enemy's.
- **REGROUP:** when a squad alone is weaker than the enemy, it disengages and moves to the other squad.
- **Reengage:** once together, the squads attack if their combined strength beats the enemy's, otherwise HOLD; a small margin prevents rapid switching.
- **Successful regroup:** the weaker squad reaches the other with survivors and the combined force wins the wave, compared with regrouping turned off.

5. Concrete Specifications and Testable Features

- **SC2 targeting and tactical decisions**
  - [built] SW0=0: nearest Roach. SW0=1: focus fire [3], the weakest Roach in range, otherwise the nearest.
  - [proposed] Squad states ATTACK, REGROUP and HOLD, from Section 4.
  - *Test:* the board's decisions match a Python reference model on recorded game states; regroup on vs. off from the same start.
- **VGA visualization of controller state**
  - [built] Units, Marine-to-target lines and the current policy.
  - [proposed] Border colour shows the stronger side (blue: Marines, red: Roaches); Marines coloured by squad state.
  - *Test:* on a paused game state, lines and colours match the decisions the board sent back.
- **UART communication**
  - [built] 115200-baud request/reply link with a checksum; a bad packet is rejected and its error code is shown on HEX.
  - *Test:* send a corrupted packet; the board rejects it and processes the next one without a reset.
- **Originality:** all HDL is written by the group. The PC bridge uses the `burnysc2` library; DeepMind's map is credited.

6. Verification Plan and Team Task Allocation

- **Marlin Lachance (lead):** existing design, architecture, squad FSM, SC2 integration.
- **Vaughn Chan (hardware):** board setup, UART and VGA testing, strength and state display.
- **Hafsa Manazir (documentation):** report, references, results.
- **Vladislav Prasko (demonstration):** two-squad scenario, integration testing, presentation.

We will also report Quartus timing and resource use and the clock cycles per decision.

[1] O. Vinyals et al., "StarCraft II: A New Challenge for Reinforcement Learning," arXiv:1708.04782, 2017.
[2] E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2nd ed., MIT Press, 2017, Fig. 1.1.
[3] D. Churchill and M. Buro, "Portfolio Greedy Search and Simulation for Large-Scale Combat in StarCraft," *IEEE CIG*, 2013.
[4] M. Stanescu, N. Barriga and M. Buro, "Using Lanchester Attrition Laws for Combat Prediction in StarCraft," *AIIDE*, 2015.